BUJUMBURA INTERNATIONAL UNIVERSITY
Faculty of BST — Software Engineering
Bachelor 2 — Course: Vue.js — Academic year 2025–2026

PROJECT REPORT

Restaurant Management Web Application

Frontend Implementation

Vue 3 · Options API · Pinia · Vue Router · Axios · Vue I18n

Student: Jerome CIRHULWIRE AJuwa
Level: Bachelor 2
Frontend repository: github.com/monamijer/restaurant-frontend
Submission date: November 6, 2026

Acknowledgements

I would like to thank my Vue.js lecturer for the course and for the guidance that made this project possible, and Bujumbura International University and the Faculty of Software Engineering (BST) for the learning environment in which it was carried out. I also thank my classmates for their feedback during the development of the application.

Abstract

This report presents the frontend of a restaurant management web application, developed as the project for the Vue.js course of the Bachelor 2 programme in Software Engineering. The application covers the journey of a restaurant customer (menu, cart, table scan, waiting queue, reservations, checkout, invoices and notifications) and provides interfaces for restaurant staff (order board, walk-in orders, reservations, queue, tables) and administrators (dashboard, categories, menu, settings, users, reports). It is a single-page application that communicates with a Django REST API.

The frontend is built with Vue 3.5 and the Options API, Vite 8, Vue Router 5, Pinia 4, Axios 1.20 and Vue I18n 11. No UI framework is used: the interface relies on custom CSS design tokens, scoped component styles, self-hosted fonts and light and dark themes. Authentication keeps the short-lived access token in memory and the refresh token in the browser storage, and a single Axios interceptor renews expired sessions once for all concurrent requests. Route guards control access by role, and the interface is available in English and French.

The code base contains 80 Vue single-file components, 8 Pinia stores and 22 API service modules. It is verified by 735 automated tests in 85 files (Vitest, Vue Test Utils and jsdom), all passing at the time of writing, with a clean ESLint run and a successful production build. Card and mobile-money payments are simulated, and the application has not been deployed to a production environment. The report describes the architecture, the implementation of each module, the testing strategy, the limitations and the possible improvements.

Keywords: Vue.js, Options API, Pinia, Vue Router, Axios, JWT, internationalisation, single-page application, restaurant.

Table of Contents

Chapter 1 — General Introduction....1
1.1 Background....1
1.2 Problem statement....1
1.3 Project objectives....1
1.4 General objective....1
1.5 Specific objectives....1
1.6 Scope of the project....2
1.7 Methodology....2
1.8 Organisation of the report....2
Chapter 2 — Project Overview and Requirements....3
2.1 Project description....3
2.2 Business context....3
2.3 Identified problems....3
2.4 Functional requirements....3
2.5 Non-functional requirements....4
2.6 User roles....4
2.7 Main application features....4
2.8 User workflows....5
Chapter 3 — Technologies and Development Environment....6
3.1 JavaScript....6
3.2 Vue.js 3....6
3.3 The Options API....6
3.4 Vite....7
3.5 Vue Router....7
3.6 Pinia....7
3.7 Axios....7
3.8 Vue I18n....7
3.9 Vitest and Vue Test Utils....8
3.10 CSS architecture....8
3.11 Development environment....8
Chapter 4 — Frontend Architecture and Design....9
4.1 Frontend architecture overview....9
4.2 Project directory structure....9
4.3 Component-based architecture....10
4.4 Vue Options API implementation....10
4.5 Component communication....11
4.6 Vue Router configuration....12
4.7 Route protection and navigation guards....13
4.8 Pinia state management....14
4.9 API service architecture....15
4.10 Authentication and token management....16
4.11 Internationalisation architecture....17
4.12 Styling and design system....18
4.13 Error handling and notifications....18
Chapter 5 — Implementation of the Application....20
5.1 Customer interface....20
5.1.1 Home page....20
5.1.2 Menu and dish details....20
5.1.3 Shopping cart....20

5.1.4 Table scanning....21
5.1.5 Queue management....21
5.1.6 Reservations....21
5.1.7 Checkout and simulated payment....21
5.1.8 Orders, tracking, invoices and notifications....22
5.1.9 Accounts and authentication pages....22
5.2 Staff interface....22
5.3 Administration interface....23
5.4 Cross-cutting implementation aspects....24
Chapter 6 — Testing and Quality Assurance....25
6.1 Testing strategy....25
6.2 Unit testing....25
6.3 Component testing....25
6.4 Store testing....25
6.5 Service testing....25
6.6 Integration testing....26
6.7 Test execution results....26
6.8 Code quality and linting....26
6.9 Known limitations of the quality process....27
Chapter 7 — Results, Limitations and Future Improvements....28
7.1 Achieved objectives....28
7.2 Implemented functionality....28
7.3 Technical achievements....28
7.4 Current limitations....28
7.5 Future improvements....29
7.6 Lessons learned....29
General Conclusion....30
References....31
Appendices....32
Appendix A — Configuration excerpts....32
A.1 ESLint rules enforcing the architecture (eslint.config.js)....32
A.2 Environment and scripts....32
Appendix B — Test helper....32
Appendix C — Verified project statistics....32
Appendix D — Screenshots to capture....33

List of Figures

Figure 1. Home page of the application (English, light theme).....5
Figure 2. Overall frontend architecture.....9
Figure 3. Component interaction (example of the menu page).....12
Figure 4. Decision flow of the global navigation guard.....13
Figure 5. Pinia stores and their main consumers.....15
Figure 6. API request and authentication flow.....17
Figure 7. Menu page with category filter and search.....20
Figure 8. Checkout page with the simulated payment panel.....22
Figure 9. Order tracking page with the status timeline.....22
Figure 10. Staff order board.....23
Figure 11. Administration dashboard.....24

List of Tables

Table 1. Functional requirements..... 3
Table 2. Non-functional requirements..... 4
Table 3. User roles and their areas..... 4
Table 4. Frontend technology stack (resolved versions)..... 6
Table 5. Directory structure of the frontend repository..... 9
Table 6. Route organisation..... 12
Table 7. Pinia stores and responsibilities..... 14
Table 8. Staff views..... 23
Table 9. Administration modules..... 24
Table 10. Verified quality results..... 26
Table 11. Test files by group..... 26
Table 12. Assessment of the specific objectives..... 28
Table 13. Environment variable and scripts..... 32
Table 14. Verified statistics of the frontend repository..... 32

List of Abbreviations

Abbreviation	Meaning
ADR	Architecture Decision Record
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
i18n	Internationalisation
JSON	JavaScript Object Notation
JWT	JSON Web Token
QR	Quick Response (code)
REST	Representational State Transfer
SPA	Single-Page Application
UI	User Interface
URL	Uniform Resource Locator
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

Chapter 1 — General Introduction

1.1 Background

Restaurants handle several flows at the same time: customers waiting for a table, tables to be assigned, orders passing from the dining room to the kitchen, payments, and the daily figures that the manager needs. When these flows are managed on paper or through informal communication, information is lost, customers wait without knowing for how long, and staff duplicate work.

A web application can bring these flows together. A customer can browse the menu on a phone, join the waiting queue, reserve a table, place an order and follow its progress. Staff see incoming orders in real time, and the manager reads reports. The project described in this report is the frontend of such a system. It runs in the browser and communicates with a separate backend through a REST API.

The project was carried out for the Vue.js examination of the Bachelor 2 programme at Bujumbura International University. The course requires Vue 3 and prohibits the use of a UI framework, so the project is also an exercise in building a complete, coherent interface from first principles.

1.2 Problem statement

Restaurant operations suffer from the following problems, which a software system can reduce:

- Customers do not know how long they will wait for a table and cannot reserve in advance without a telephone call.
- Orders are taken verbally or on paper, which causes errors and delays between the dining room and the kitchen.
- Staff cannot see the state of all orders and tables on a single screen.
- Managers have no immediate access to sales figures, popular dishes or user management.
- Customers and staff use different languages, and an interface that is difficult to use on a phone excludes many customers.

The question addressed by this report is: **how can a Vue.js single-page application, built without a UI framework, provide a maintainable, accessible and tested interface for these three groups of users?**

1.3 Project objectives

1.4 General objective

To design and implement the frontend of a restaurant management application with Vue 3 and the Options API, connected to a REST backend, covering customer, staff and administration needs.

1.5 Specific objectives

- Organise the interface as reusable components and lazily loaded views.
- Manage shared state with Pinia and keep request-specific state local to components.
- Control navigation and access with Vue Router, navigation guards and role metadata.

- Communicate with the API through a single Axios instance with automatic token renewal.
- Provide the interface in English and French with Vue I18n.
- Build a consistent, accessible, responsive design system with custom CSS, without any UI framework.
- Verify the application with automated unit, component, service and routing tests.

1.6 Scope of the project

The work covers the frontend repository (`restaurant-frontend`). The backend repository (`restaurant-backend`, Django REST Framework) is the data and business-rule provider; it is discussed only where required to explain the API contract, authentication and data exchange. It is not the subject of this report.

The scope has deliberate limits. Payments by card and mobile money are simulated by the backend; no real payment gateway is connected. The application has not been deployed to a public production server and is run locally. The in-app scanning of QR codes with the camera is not implemented: a table QR code opens the application through a URL. These limits are repeated where they matter in the following chapters.

1.7 Methodology

The application was developed incrementally, one feature at a time, from the foundation (configuration, router, API layer, design tokens) towards the customer features, and then the staff and administration modules. The repository history contains 92 commits between 14 October and 5 November 2026. Several rules were fixed early and applied throughout:

- **Architecture decisions were recorded** and enforced where possible: the Options API and the absence of `v-html` are enforced by ESLint rules, and the equality of the English and French translation keys is enforced by a test.
- **Every feature comes with tests.** The checks `npm run lint`, `npm test` and `npm run build` are run before a change is committed.
- **The backend is the authority.** The frontend never computes or sends a price and never decides on its own who may perform an action; it only improves the experience.
- **Mobile-first design** with CSS design tokens shared by all components.

For this report, the source code was examined directly: the router, stores, services, views and tests were read, and the test suite, the linter and the production build were executed. Where the project documentation and the code differed, the code was taken as the reference.

1.8 Organisation of the report

Chapter 2 describes the project and its requirements. Chapter 3 presents the technologies used and their role in the application. Chapter 4, the central chapter, explains the frontend architecture: components, routing, state, API access, authentication, internationalisation and styling. Chapter 5 describes the implemented modules. Chapter 6 presents the testing strategy and the verified results. Chapter 7 discusses the results, the limitations and future improvements. A general conclusion, the references and the appendices follow.

Chapter 2 — Project Overview and Requirements

2.1 Project description

The application is a restaurant **table, queue and online ordering system**. Its frontend is a Vue 3 single-page application that gives each type of user a dedicated set of pages. Customers can consult the menu without an account; creating an account unlocks ordering, reservations, invoices and notifications. Staff and administrators sign in with accounts that carry a role, and they reach protected areas under the `/staff` and `/admin` paths.

2.2 Business context

The system models a single restaurant. The restaurant's name, currency, time zone and opening rules are stored as settings on the backend and displayed by the frontend (the `restaurant` store). Customers may dine in, take away or request delivery. A dine-in customer may be linked to a table by scanning the QR code placed on it. Staff receive orders and move them through preparation, and the administrator manages the catalogue, the users and the settings.

2.3 Identified problems

From the problem statement, the following needs were derived: a public, searchable menu; an ordering flow that works on a phone; a visible queue with a position and an estimated wait; reservations by date, party size and time slot; order tracking for customers; an order board for staff; and management and reporting tools for administrators.

2.4 Functional requirements

Table 1. Functional requirements

Id	Requirement	Actor
FR1	Browse and filter the menu by category, search text and page.	Visitor
FR2	Manage a cart (quantities, notes) that survives a page reload.	Visitor
FR3	Register, sign in, sign out, recover a forgotten password, change the password and edit the profile.	Visitor, client
FR4	Link a table through a QR-code URL and place dine-in orders for it.	Client
FR5	Join and leave the waiting queue; view queue length and estimated wait.	Visitor, client
FR6	Search free time slots, create and cancel reservations.	Client
FR7	Check out an order (dine-in, takeaway, delivery) and pay it with a simulated gateway.	Client
FR8	Follow an order in real time and read invoices and notifications.	Client
FR9	Receive and advance orders, take walk-in orders, confirm cash payments.	Staff
FR10	Manage reservations, queue entries and tables, and download table QR codes.	Staff
FR11	Manage categories, menu items with images, settings and users.	Administrator
FR12	Read a dashboard and sales reports, and export them to CSV.	Administrator

2.5 Non-functional requirements

Table 2. Non-functional requirements

Quality	Requirement	How it is addressed
Maintainability	Predictable structure, one coding model.	Options API only (ESLint-enforced), one service file per backend area, shared base components.
Security	Limit the impact of stolen tokens and XSS.	Access token in memory, rotating refresh token, `safeRedirect`, no `v-html`, validated cart storage.
Accessibility	Usable by keyboard and assistive technologies.	Real buttons and links, labels, `aria-live` regions, focus moved on navigation, 44 px targets, contrast tests.
Responsiveness	Usable on a phone.	Mobile-first CSS with `min-width` media queries.
Internationalisation	English and French.	Vue I18n with 16 namespaces and a parity test.
Performance	Fast first load.	All views lazy-loaded; main bundle 275 kB (95 kB gzip).
Testability	Automated verification.	735 tests; the network is replaced by a fake adapter.

2.6 User roles

The backend defines three roles, mirrored in `src/utils/roles.js`: `client`, `staff` and `admin`. Visitors who are not signed in have no role. The frontend uses the role to decide which links to display and which routes to allow; the backend remains the authority and checks every request again.

Table 3. User roles and their areas

Role	Landing page	Main areas
Visitor	/	Menu, cart, queue summary, scan page, sign in, register, legal pages.
Client	/client/account	Orders, reservations, invoices, notifications, profile, checkout.
Staff	/staff	Order board, walk-in order, reservations, queue, tables; also client pages.
Admin	/admin	Dashboard, categories, menu, settings, users, reports; also staff pages.

2.7 Main application features

The application has 43 route declarations, of which 28 require a signed-in user. The customer area is the largest, followed by the administration area (13 files) and the staff area (7 views). The features are described module by module in Chapter 5.

2.8 User workflows

Three workflows summarise the use of the application.

Dine-in order. The customer scans the QR code on a table; the application stores the table token for the browser session and shows a banner. The customer fills the cart from the menu and checks out; the order is created for the table. The customer follows its status on a timeline that is refreshed every ten seconds, then pays online (simulated) or at the counter.

Reservation. The signed-in customer chooses a date and a party size, the application asks the backend for free slots, the customer selects a time, and the reservation is created. It appears in the reservation list, where it can be cancelled, and staff see it in their own list.

Order processing. A staff member opens the order board, filters by status, opens an order and advances it through the sequence `pending` → `confirmed` → `preparing` → `ready` → `completed`. For a cash payment, the staff member confirms the receipt of the money.



Figure 1. Home page of the application (English, light theme)

Chapter 3 — Technologies and Development Environment

This chapter presents the technologies of the frontend. For each one, it states what it is, why it is useful and how it is used in this project. The versions are those resolved in `package-lock.json`.

Table 4. Frontend technology stack (resolved versions)

Technology	Installed version	Role
Vue	3.5.44	Component framework
Vite (with `@vitejs/plugin-vue` 6.0.10)	8.3.4	Development server and production build
Vue Router	5.4.0	Client-side routing
Pinia	4.1.0	Shared state
Axios	1.20.0	HTTP client
Vue I18n	11.4.14	Translations
Vitest	5.0.4	Test runner
@vue/test-utils	2.5.1	Component mounting in tests
jsdom	29.1.1	Browser simulation for tests
ESLint (with eslint-plugin-vue 10.11.1)	10.12.0	Static analysis
@fontsource-variable/fraunces, /inter	5.3.0	Self-hosted fonts

3.1 JavaScript

JavaScript is the language of the browser and of Vue. The project uses modern JavaScript (ES modules, `async`/`await`, optional chaining) and no TypeScript. The language is used for components, stores, services and pure helpers in `src/utils` (money, dates, errors, roles, redirect handling). Keeping business-neutral logic in small pure functions makes it easy to test: for example, `src/utils/money.js` converts amounts to integer cents to avoid floating-point errors in cart totals.

3.2 Vue.js 3

Vue is a progressive framework for building user interfaces from components. Its reactivity system updates the DOM automatically when data changes, so the code describes the state and the template describes how the state is displayed. In this project Vue renders the whole interface: 80 single-file components (`.vue`) combine a template, a script and scoped styles. Directives such as `v-if`, `v-for`, `v-model` and `:class` express the interface declaratively, and a custom directive, `v-reveal`, adds a reveal-on-scroll effect. `v-html` is forbidden by the linter, which removes a common XSS vector.

3.3 The Options API

Vue 3 offers two ways of writing components. The Composition API organises code in functions (`setup()` and `<script setup>`); the Options API organises it in a component object with named

sections: `data`, `computed`, `watch`, `methods` and lifecycle hooks such as `created`. This project uses the Options API exclusively. The choice is enforced by three ESLint rules (Appendix A.1), so a component written in the Composition style fails the lint check.

The reasons are practical. A component reads as a single object whose shape is predictable and searchable; Pinia stores and router helpers are connected in the same way in every file (`mapState`, `mapActions`); and tests read `wrapper.vm` directly. The consequence is that logic shared between components cannot be a composable function: it becomes a Pinia store, a mixin (`formErrors`, `cartMoney`) or a pure helper in `src/utils`.

3.4 Vite

Vite is a build tool that serves source modules directly during development and bundles them with a production build. In the project it provides the development server on port 5173 (`strictPort`, because the backend accepts only that origin through CORS), the `@` alias for `src/`, hot module replacement, and the production bundle. The verified production build succeeds, splits each view into its own chunk through dynamic imports, and produces a main chunk of about 275 kB (95 kB compressed). The API address is injected at build time through the `VITE_API_BASE_URL` environment variable.

3.5 Vue Router

Vue Router maps URLs to views, supports nested and parameterised routes, and exposes navigation guards. The project declares 43 routes in `src/router/routes.js`; every view is loaded lazily, and route `meta` fields (layout, title key, authentication, roles) drive the page shell and the guards. Section 4.6 and 4.7 describe the configuration.

3.6 Pinia

Pinia is the official state-management library for Vue. A store holds state, getters (derived values) and actions (operations, including asynchronous ones). The project defines eight stores in the options style and consumes them with `mapState` and `mapActions`. Pinia is used only for state that is read by several routes and must remain consistent between them (Section 4.8).

3.7 Axios

Axios is an HTTP client with interceptors, request configuration and automatic JSON handling. A single configured instance (`src/services/api.js`) carries the base URL, the timeout, the authentication header, the transparent renewal of expired sessions and the conversion of every failure into a uniform `ApiError`. Twenty-two service modules build on this instance.

3.8 Vue I18n

Vue I18n provides translation functions, pluralisation and number and date formatting. The application supports English and French through 16 namespaces of messages per language, with the language switcher stored in the `ui` store (Section 4.11).

3.9 Vitest and Vue Test Utils

Vitest is a test runner designed for Vite projects; Vue Test Utils mounts components so that tests can interact with them; jsdom simulates a browser in Node.js. Together they run 735 tests in 85 files. Tests replace the network by a fake Axios adapter (`fakeServer` in `tests/helpers/http.js`) and mount components with the real Pinia and translations through a `mountWithApp` helper. Chapter 6 reports the results.

3.10 CSS architecture

The specification forbids UI frameworks, so styling is written in plain CSS: custom properties in `src/assets/styles/tokens.css` (typography, spacing on a 4 px grid, colours, radii, motion, target size), a small `base.css` with global utility classes (`container`, `stack`, `cluster`, `muted`, `eyebrow`, `visually-hidden`) and scoped styles in each component. The light and dark themes are two sets of colour values selected by a `data-theme` attribute on the root element. Section 4.12 details the system.

3.11 Development environment

The project requires Node.js 22.13 or later and npm. The commands `npm run dev`, `npm run build`, `npm run preview`, `npm test` and `npm run lint` start the development server, build and preview the production bundle, run the tests and run ESLint. The report's verification used Node.js 22 with a clean `npm ci`. The repository has no continuous-integration pipeline; the three checks are run manually before each commit.

Chapter 4 — Frontend Architecture and Design

4.1 Frontend architecture overview

The frontend is organised in four layers. The **presentation layer** (layouts, views and components) renders the interface. The **routing layer** decides which view is shown and whether the visitor may see it. The **state layer** (Pinia) holds the data that several pages share. The **data access layer** (services, the Axios instance and the session module) communicates with the backend. Vue I18n and the design tokens are transversal: every view uses them. Figure 2 summarises these relations as they exist in the code.

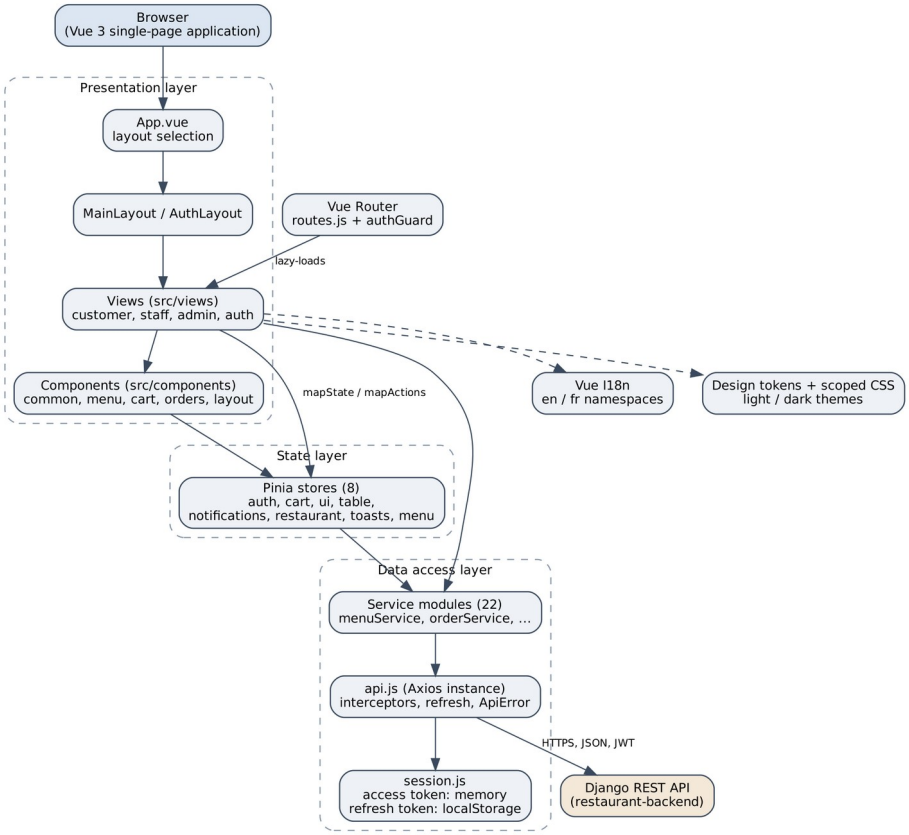


Figure 2. Overall frontend architecture

The entry point `src/main.js` creates the application, installs Pinia, the i18n instance and the router, and registers the session-expiry handler and the synchronisation of the cart between browser tabs. `App.vue` reads `route.meta.layout` to choose between `MainLayout` (header, table banner, footer) and `AuthLayout` (a centred card for sign-in pages), and loads the restaurant settings and the notifications when a user is signed in.

4.2 Project directory structure

Table 5. Directory structure of the frontend repository

Path	Content
`src/main.js`, `App.vue`, `config.js`	Entry point, root component, constants (API URL, timeouts, currency, limits, locales).

Path	Content
src/router/	routes.js, guards.js, title.js (document title and focus), expiry.js.
src/store/	Eight Pinia stores.
src/services/	api.js, session.js and 22 service modules, one per backend area.
src/views/	Customer pages, plus staff/, admin/, account/, auth/, errors/, legal/.
src/components/	common (14 shared base components), layout, menu, cart, orders, home, account, staff, admin.
src/layouts/	MainLayout.vue and AuthLayout.vue.
src/mixins/, src/directives/	Two mixins (formErrors, cartMoney) and the v-reveal directive.
src/i18n/	i18n setup and locales/{en,fr}/<namespace>.json.
src/utils/	Pure helpers: money, dates, errors, roles, redirect, menu query, orders.
src/assets/styles/	tokens.css and base.css.
tests/	unit/ (27 files), components/ (26), views/ (25), services/ (7), helpers/, setup.js.

The folders separate responsibilities rather than technical file types alone: a view never calls Axios directly, it calls a service; a service never touches the DOM; a utility never imports a store. This rule keeps each layer testable in isolation.

4.3 Component-based architecture

The interface is composed of three kinds of components. **Views** correspond to routes and orchestrate a page: they read the route, call services and pass data down. **Feature components** (MenuFilters, MenuItemCard, OrderTimeline, PaymentPanel, QuantityStepper, and others) implement a part of a page. **Base components** in components/common (for example BaseButton, BaseInput, BaseModal, BaseBadge, BaseSkeleton, BasePagination, EmptyState, ConfirmDialog, ToastHost) implement the visual vocabulary of the design system and are reused by all areas. Because the base components accept props and emit events only, replacing a button style or a modal behaviour is done in one place.

4.4 Vue Options API implementation

Every component is a default-exported object. The MenuView component, abbreviated below, shows the typical structure: local state in data, derived values in computed, a watch on the route, a lifecycle hook, and methods that call the store actions and services.

Listing 4.1 — Structure of src/views/MenuView.vue (abridged)

```
export default {
  name: 'MenuView',
  data() {
    return { items: [], count: 0, loading: false, failed: false,
             requestId: 0 } // Only the latest request may update the page.
  },
  computed: {
    ...mapState(useMenuStore, ['categories']),
    filters() { return parseMenuQuery(this.$route.query) },
  },
  watch: {
    '$route.fullPath'() { if (this.$route.name === 'menu') this.fetchItems() },
  },
  created() { this.loadCategories(); this.fetchItems() },
  methods: {
    ...mapActions(useMenuStore, ['loadCategories']),
    async fetchItems() { /* see below */ },
  },
}
```

The method `fetchItems` illustrates a pattern used by every list page. Each call increments `requestId` and compares it with the value captured at the start; an answer that arrives after a newer request was issued is ignored. This prevents an old, slow response from overwriting a newer result when the visitor changes filters quickly.

Listing 4.2 — Stale-response guard (abridged from MenuView.vue)

```
async fetchItems() {
  const id = ++this.requestId
  this.loading = true
  this.failed = false
  try {
    const page = await menuService.items(toApiParams(this.filters))
    if (id !== this.requestId) return
    this.items = page.results
    this.count = page.count
  } catch (error) {
    if (id !== this.requestId) return
    this.failed = true
  } finally {
    if (id === this.requestId) this.loading = false
  }
}
```

The address of the page is the source of truth for the filters: `filters` is a computed property derived from `$route.query`, and any change of the filters updates the URL, which triggers the watcher. As a result, a filtered menu page can be bookmarked, shared and navigated with the browser's back button.

4.5 Component communication

Communication follows three mechanisms. **Props down, events up** is used between a parent and its children: `MenuView` passes `filters` and `categories` to `MenuFilters`, which emits an update event. **Pinia** connects distant components, such as the header, the cart page and the checkout, which all read the `cart` store. **The router** carries state that must survive reloads and sharing (filters, page

number, redirect target). Events are declared with `emits`, and the rule `vue/require-explicit-emits` makes the linter reject undeclared events; `vue/require-default-prop` requires defaults for optional props. Figure 3 shows the relations.

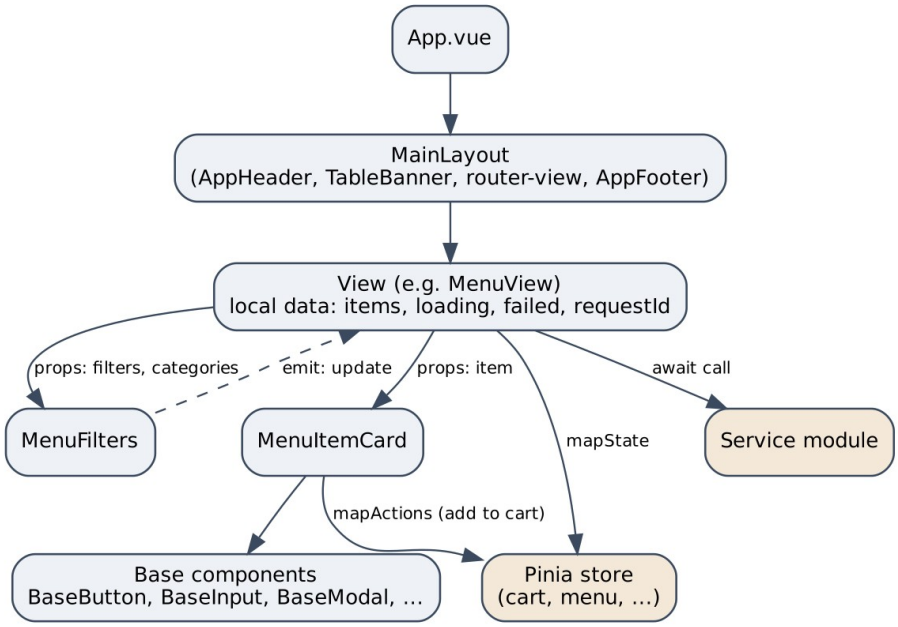


Figure 3. Component interaction (example of the menu page)

4.6 Vue Router configuration

Routes are declared as an array in `src/router/routes.js`. Each route has a name, a lazily imported view and a `meta` object that drives the application shell and the guards. The meta fields are documented in the file itself: `layout` (`main` or `auth`), `titleKey`, `requiresAuth`, `roles` and `guestOnly`.

Listing 4.3 — A public route in src/router/routes.js

```
{
  path: '/menu',
  name: 'menu',
  component: () => import('@/views/MenuView.vue'),
  meta: { layout: 'main', titleKey: 'menu.title' },
},
```

Table 6. Route organisation

Area	Examples of paths	Access
Public	`/`, `/menu`, `/menu/:id`, `/cart`, `/queue`, `/scan`, `/privacy`, `/terms`	Everybody
Authentication	`/login`, `/register`, forgot- and reset-password pages	`guestOnly` for sign-in pages
Client	`/checkout`, `/orders`, `/orders/:id`, `/reservations`, `/client/account`	`requiresAuth`
Staff	`/staff`, order board, walk-in order, reservations, queue, tables	roles: staff, admin

Area	Examples of paths	Access
Admin	`/admin`, dashboard, categories, menu, settings, users, reports	roles: admin
Errors	`forbidden`, unknown address page	Everybody

The development-only `/styleguide` route is registered only when `import.meta.env.DEV` is true. After each navigation, `title.js` sets the document title from the translated `titleKey` and moves the keyboard focus to the main region, except when only the query string changed; this helps screen-reader and keyboard users to notice that the page changed. Page titles also follow the current language.

4.7 Route protection and navigation guards

A single global guard, `authGuard`, runs before every navigation. It first waits for the stored session to be restored (once, and shared by concurrent callers), then applies three rules in order. Figure 4 shows the decision flow; Listing 4.4 shows the code.

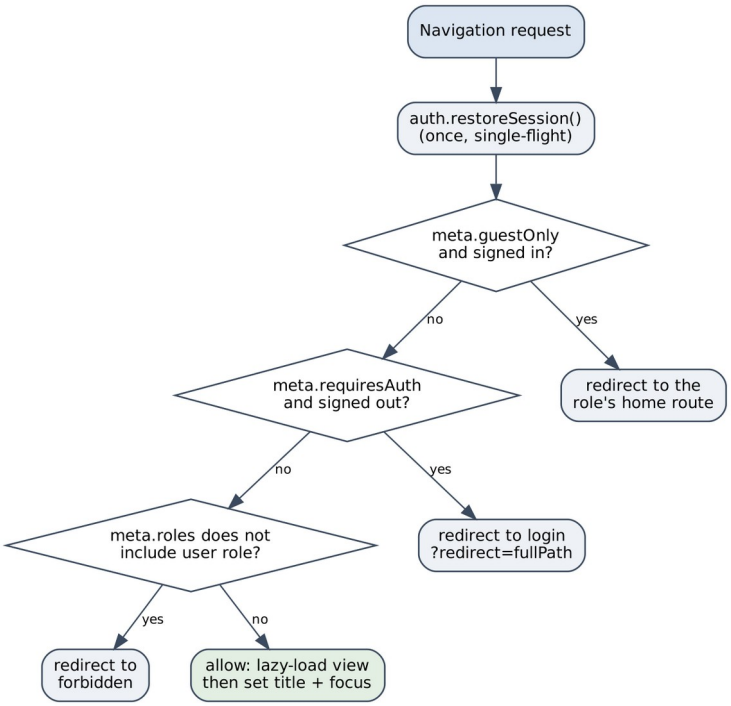


Figure 4. Decision flow of the global navigation guard

Listing 4.4 — src/router/guards.js

```
export async function authGuard(to) {
  const auth = useAuthStore()
  await auth.restoreSession()
  if (to.meta.guestOnly && auth.isAuthenticated) {
    return { name: homeRouteFor(auth.role) }
  }
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.roles && !to.meta.roles.includes(auth.role)) {
    return { name: 'forbidden' }
  }
  return true
}
```

The comment in the source states the design principle: the guard improves the experience, because the visitor never sees a page that cannot be used, but it is not security. Every API request is checked again by the backend, so modifying the frontend cannot grant access to data. The role information (`meta.roles`) mirrors permissions that exist on the backend. After a successful sign-in, the visitor is sent back to the page requested, through the `redirect` query parameter, which is validated (Section 4.10).

4.8 Pinia state management

The project applies one criterion to decide whether a piece of state becomes a store: it must be read by more than one route and stay consistent across them. Everything else remains local to a component. Table 7 lists the eight stores with their responsibilities. All stores are written in the options style (`state`, `getters`, `actions`).

Table 7. Pinia stores and responsibilities

Store	State	Responsibilities and notable behaviour
`auth`	`user`, `restored`	Getters `isAuthenticated`, `role`, `isStaff`, `isAdmin`, `fullName`. Actions `restoreSession` (single-flight), `login`, `register`, `logout`, `expireSession`.
`cart`	`lines`	Persisted in `localStorage`; each stored line is validated on reading; totals are display estimates in integer cents; the order payload is built by a getter.
`ui`	theme, locale	Theme (follows the system on the first visit) and language; persisted; only the switcher applies a locale.
`table`	scanned table	Kept in `sessionStorage` so that the scanned table survives a reload of the menu page.
`notifications`	items, unread count	Polled every 30 seconds by the header; polling is skipped when the tab is hidden.
`restaurant`	settings	Name, currency and time zone read by the header, home page, reservation form and checkout.
`toasts`	queue	One queue and one host (`ToastHost`); any page can push a message.
`menu`	categories	Cached for the filters and the cart.

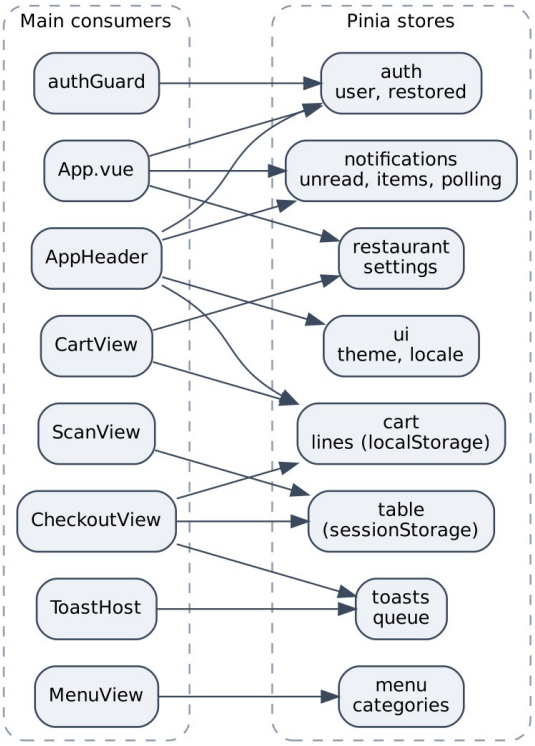


Figure 5. Pinia stores and their main consumers

List pages deliberately do not use a store: each keeps its own `items`, `loading`, `failed` and `requestId`. The request lifecycle and the stale-response guard are then visible next to the template that uses them, and a list page can be copied to a new route without dragging a store along. For the same reason, stores are not used as a data cache: two pages that need the same list (the administrator's and the staff's lists of orders) call their own service. The cart store illustrates defensive design: since `localStorage` can be edited by hand, every stored line is checked (identifier, name length, price pattern, quantity bounds) before it is trusted, and the cart is truncated to a maximum number of lines.

4.9 API service architecture

All backend access goes through one Axios instance defined in `src/services/api.js`. Twenty-two service modules (for example `menuService`, `queueService`, `authService`, `staffOrderService`, `adminUserService`) export plain objects whose methods wrap one endpoint each and return the data. A service handles URLs, parameters and request bodies; it contains no interface logic. Two request options control the interceptors: `skipAuth` (public endpoints and sign-in) and `skipRefresh` (a 401 must not trigger a renewal, for example after a wrong password).

Listing 4.5 — Excerpt of src/services/queueService.js

```
export const queueService = {
  async summary() {
    const { data } = await api.get('/queue/summary/', PUBLIC)
    return data
  },
  async join({ partySize, phone }) {
    const body = { party_size: partySize }
    if (phone) body.phone = phone
    const { data } = await api.post('/queue/', body)
    return data
  },
}
```

The backend returns errors in a common envelope containing a success flag, a message, field errors and a code. The helper `toApiError` converts any Axios failure (including the absence of a network answer) into an `ApiError` that carries these fields; the application maps backend codes to translated messages. A mixin, `formErrors`, applies field errors to forms.

4.10 Authentication and token management

The backend issues JSON Web Tokens: a short-lived access token (15 minutes) and a refresh token (7 days) that is rotated on each use, the previous one being blacklisted. These values were verified in the backend settings. The frontend stores them differently:

- The **access token** lives in a module-level variable in `session.js`; it is never written to storage, and a page reload loses it.
- The **refresh token** is kept in `localStorage` under `restaurant.refresh` (with an in-memory fallback when storage is unavailable). At startup the guard calls `restoreSession`, which exchanges it for a new pair and loads the user.

Listing 4.6 — Excerpt of src/services/session.js

```
let accessToken = null
let fallbackRefreshToken = null // Used when localStorage is unavailable.

export const session = {
  getAccessToken() { return accessToken },
  setTokens({ access, refresh }) { /* memory + localStorage */ },
  clear() { /* forget both tokens */ },
}
```

The request interceptor adds the `Authorization: Bearer` header unless `skipAuth` is set. The response interceptor handles a `401` answer in a controlled way. If a refresh token exists, and the request was not already retried and does not opt out, the interceptor calls `refreshTokens()` and replays the request once with the new token. `refreshTokens` stores the pending promise in a module variable, so requests that fail at the same time share a single renewal; this matters because the backend rotates refresh tokens and two parallel renewals would invalidate each other. If the renewal fails, the session is cleared and the application is told once (a `WeakSet` remembers the announced failure), which displays a message and sends the visitor to the sign-in page.

Listing 4.7 — Single-flight refresh (abridged from src/services/api.js)

```
let refreshInFlight = null
export function refreshTokens() {
  if (!refreshInFlight) {
    const refresh = session.getRefreshToken()
    if (!refresh) return Promise.reject(toApiError(new Error('No refresh token')))
    refreshInFlight = api
      .post('/auth/refresh/', { refresh }, { skipAuth: true, skipRefresh: true })
      .then(({ data }) => {
        session.setTokens({ access: data.access, refresh: data.refresh })
        return data.access
      })
      .catch((error) => { session.clear(); throw toApiError(error) })
      .finally(() => { refreshInFlight = null })
  }
  return refreshInFlight
}
```

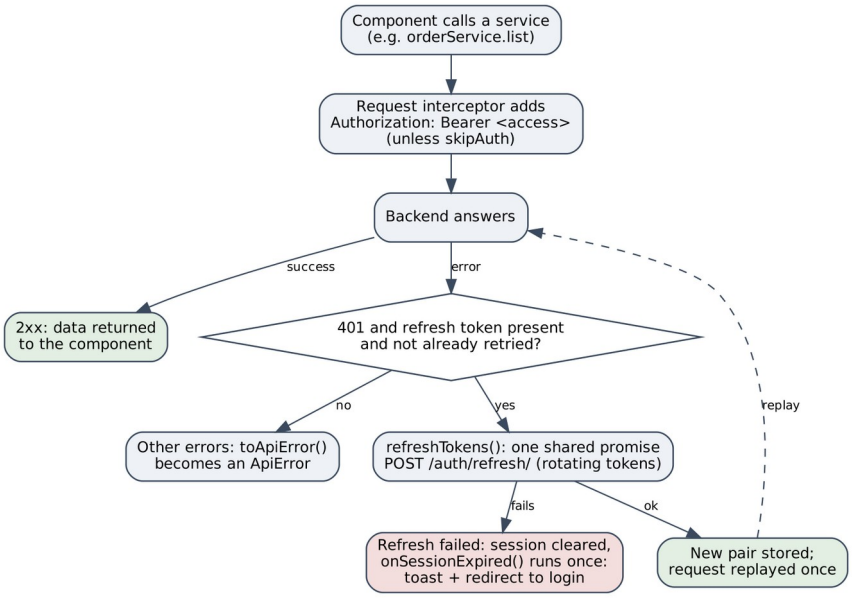


Figure 6. API request and authentication flow

Redirection after sign-in is protected against open redirects: the `redirect` query value is passed through `safeRedirect` (`src/utils/redirect.js`), which accepts only internal paths and rejects absolute URLs, protocol-relative addresses (`//`) and backslashes. Signing out calls the logout endpoint to blacklist the refresh token and clears the local session even if the call fails.

The combination limits the damage of token theft. A stolen access token stops working after 15 minutes; a stolen refresh token works only once because of rotation. The cost is one extra request when the page is reloaded. The trade-off is acknowledged: `localStorage` remains readable by any script running in the page, which is why the project avoids `v-html` and validates stored data.

4.11 Internationalisation architecture

Translations are stored in `src/i18n/locales/{en,fr}/<namespace>.json`, with 16 namespaces per language (for example menu, orders, billing, staff, admin). They are merged in `src/i18n/index.js` and exposed through `$t`. Several decisions are worth noting.

- **Default language.** English is the default for everybody; the browser language is deliberately not consulted, so two visitors see the same language until one of them opens the switcher. The choice is stored in `localStorage` under `restaurant.locale`.
- **Mode.** `createI18n` is called with `legacy: false` (the current mode of Vue I18n 11) and `globalInjection: true`, which exposes `$t`, `$n` and `$d` on `this` and in templates. The components remain in the Options API.
- **Parity.** A unit test (`tests/unit/i18n.test.js`) requires both languages to define exactly the same keys, with the same placeholders and no empty string, so a missing translation fails the build before reaching users.
- **Plurals** use the pipe syntax and are called with a count, for example `$t('queue.waiting', { count: n }, n)`.

Listing 4.8 — Vue I18n configuration (src/i18n/index.js)

```
export const i18n = createI18n({
  legacy: false,
  globalInjection: true,
  locale: detectLocale(),
  fallbackLocale: DEFAULT_LOCALE,
  messages: { en, fr },
})
```

Limitation. The interface text is translated, but the content that comes from the backend (menu item names and descriptions, and notification messages) is stored in a single language by the backend and is not translated.

4.12 Styling and design system

The project avoids UI frameworks and Tailwind CSS, as required, and still needs a coherent appearance. The solution is a set of **design tokens**: CSS custom properties declared in `tokens.css` and used by every component. Components never hard-code a colour, size or motion value. The file defines typography (Fraunces for headings and Inter for text, both self-hosted so that no external font request is made), a fluid type scale with `clamp()`, spacing on a 4 px grid, and the colours of two themes selected by `:root[data-theme='light']` and `:root[data-theme='dark']`. The theme follows the system preference on the first visit and is then stored; a small script in `public/theme-init.js` applies it before the first paint to avoid a flash.

Component styles are `scoped`, so selectors stay local to the component; the few cross-cutting rules live in `base.css`. Media queries use `min-width` only, in a mobile-first approach, and interactive targets use a minimum size of 44 px (`--target-size`). Quality is protected by tests: `tests/unit/tokens-contrast.test.js` walks the colour tokens and checks that text and surface combinations meet the WCAG contrast ratios in both themes. The page transition and the modal animation are disabled for users who request reduced motion, and status changes never rely on colour alone: a badge carries a text label, an icon or an `aria-live` announcement.

4.13 Error handling and notifications

Errors are handled at four levels. Network and server failures are normalised into `ApiError` by the interceptor. Views catch errors in their own methods and set a `failed` flag that displays an error state with a retry action, or apply field errors to a form (`formErrors` mixin). The expiry of a session

produces one message and one redirect. Finally, user feedback uses the `toasts` store and the `ToastHost` component (an `aria-live` region) for transient messages, and `ConfirmDialog` for destructive or financial actions such as leaving the queue or confirming a cash payment. Loading states use `BaseSkeleton`, and empty lists use `EmptyState`, so that the three states (loading, empty, failed) are explicit on every list page.

Chapter 5 — Implementation of the Application

This chapter describes the modules that exist in the repository. Each module was verified in the source code and is covered by tests. For each module, the relevant views and components, the main logic, the state management, the data exchange and the validation are presented. Screenshot placeholders mark the screens that should be captured to complete the report.

5.1 Customer interface

5.1.1 Home page

`HomeView` presents the restaurant through the `restaurant` store (name and settings) and a `HeroCarousel` component in `components/home`, with scroll-reveal animations provided by the `v-reveal` directive, which respects the reduced-motion preference. The page is public, and its text is fully translated.

5.1.2 Menu and dish details

Views and components: `MenuView`, `MenuItemView`, `MenuFilters`, `MenuItemCard`, `BasePagination`, `BaseSkeleton`, `EmptyState`. **State:** the filters (category, search text, page) live in the URL query; the categories are cached in the `menu` store; the items are local to the view. **Data:** `menuService.items` receives parameters built by `toApiParams`. **Validation:** `parseMenuQuery` in `src/utils/menuQuery.js` whitelists the values read from the URL, so a manipulated address cannot send arbitrary parameters. **Behaviour:** the stale-response guard (Listing 4.2), an explicit loading, empty and failed state, a status line announced to assistive technologies (`statusText`), and a return to page 1 when the requested page no longer exists.

SCREENSHOT PLACEHOLDER

To capture: the /menu page with several dishes, one category selected and a search term entered

Figure 7. Menu page with category filter and search

5.1.3 Shopping cart

Views and components: `CartView`, `AddToCartButton`, `QuantityStepper`, `CartLink` (in the header). **State:** the `cart` store (Section 4.8). Lines contain a menu item identifier, a name and a price snapshot, a quantity and an optional note, bounded by `MAX_CART_LINES`, `MAX_LINE_QUANTITY` and `MAX_NOTE_LENGTH` from `config.js`. The cart is persisted in `localStorage` under `restaurant.cart`, and `main.js` keeps several tabs synchronised. **Money:** amounts are decimal strings from the API; the cart computes totals in integer cents through `src/utils/money.js` and the `cartMoney` mixin, and labels them as estimates, because the

backend recomputes every price when it creates the order. The client never sends a price: `orderItems` produces only a menu item identifier, a quantity and a note.

5.1.4 Table scanning

Each table of the restaurant has a QR code that encodes a URL of the form `/scan?t=<token>`. `ScanView` reads the token from the query, asks the backend to resolve it (`tableService`), and stores the table in the `table` store, which uses `sessionStorage` so that the table survives a reload of the menu page. `TableBanner`, part of `MainLayout`, then displays the linked table on every page. The checkout uses the stored table for dine-in orders. This is a URL-based scan: the application itself does not access the camera; the visitor uses the phone's camera application or any QR reader.

5.1.5 Queue management

`QueueView` displays a public summary (number of waiting parties and the estimated wait, fetched with `skipAuth`) and, for a signed-in customer, the current ticket with its position and the number of parties ahead (a pluralised message). A customer who is not in the queue can join with a party size (a `select` bound with `v-model.number`) and an optional phone number; a customer who is in the queue can leave it after a `ConfirmDialog`. A visitor who is not signed in is invited to sign in. Staff manage the same queue from `StaffQueueView` (Section 5.2).

5.1.6 Reservations

`ReservationNewView` implements a two-step flow: the customer chooses a date and a party size, the application asks the backend for the free slots, and the customer selects a time and confirms. The default date is computed in the restaurant's time zone (`todayIn(settings.timezone)`), not the browser's. A watcher forgets the slots whenever the date or party size changes, so a displayed time always corresponds to the current search. The party size is limited to `MAX_PARTY_SIZE`. `ReservationsView` lists the customer's reservations and allows cancellation.

5.1.7 Checkout and simulated payment

`CheckoutView` converts the cart into an order. The customer chooses dine-in, takeaway or delivery; a dine-in order uses the scanned table. The view reads the cart through `mapState`, sends `orderItems` to `orderService`, and clears the cart once the order is created. Errors are shown with the form-error mixin and toasts.

Payment is handled by `PaymentPanel` (`components/orders`) with `paymentService`. The customer may choose card, mobile money or cash. **Card and mobile-money payments are simulated:** the interface displays a demonstration notice, no real gateway is contacted, and a special test token (`tok_declined`) produces a declined payment so that the failure path can be tested. Cash payments are confirmed by a staff member. The legal pages state that no real money is handled. The backend only accepts simulated payments when it is configured to do so.

SCREENSHOT PLACEHOLDER

To capture: the /checkout page with a filled cart, showing the payment method choice and the demonstration notice

Figure 8. Checkout page with the simulated payment panel

5.1.8 Orders, tracking, invoices and notifications

`OrdersView` lists the customer's orders, and `OrderView` tracks one order. `OrderTimeline` shows the status sequence (`pending`, `confirmed`, `preparing`, `ready`, `completed`), with a distinct state for `cancelled`; a `BaseBadge` carries the status as text and a tone, so colour is never the only signal. The view refreshes the order every 10 seconds (`POLL_INTERVAL_MS`), but not when the tab is hidden or the order is settled, and a 404 answer displays a not-found state. Cancellation is offered only while the order is pending and unpaid. `InvoicesView` and `InvoiceCard` display the invoices, and `NotificationsView` displays the messages that the `notifications` store polls for the header bell (`NotificationBell`).

SCREENSHOT PLACEHOLDER

To capture: an order page (/orders/:id) in the "preparing" state

Figure 9. Order tracking page with the status timeline

5.1.9 Accounts and authentication pages

`LoginView`, `RegisterView`, `ForgotPasswordView` and `ResetPasswordView` use `AuthLayout` and are `guestOnly`. `AccountView`, `AccountProfileView` and `AccountPasswordView` let a signed-in user edit the profile and change the password. Forms use `BaseInput` with labels tied to inputs and error messages, validate on submission, and display the field errors returned by the backend. After registration, the new customer is signed in directly.

5.2 Staff interface

The staff area (`/staff`) is restricted to the staff and admin roles and has its own navigation component, `StaffNav`. Seven views were implemented:

Table 8. Staff views

View	Purpose	Main data and logic
`StaffHomeView`	Landing page of the area.	Entry points to the tools.
`StaffOrdersView`	Order board.	Lists orders with status filters; refreshed periodically through `staffOrderService`.
`StaffOrderView`	Order detail.	Advances the order through `SEQUENCE = [pending, confirmed, preparing, ready, completed]`; confirms a cash payment after a `ConfirmDialog` (`staffPaymentService`).
`StaffOrderNewView`	Walk-in order.	Creates an order for a customer without an account.
`StaffReservationsView`	Reservations.	Lists and manages reservations.
`StaffQueueView`	Waiting queue.	Lists and manages queue tickets.
`StaffTablesView`	Tables and QR codes.	Lists tables; downloads the QR code of a table (PNG) and a printable sheet (PDF), both produced by the backend and received as binary data.

The frontend displays only the next step of an order, but the legal transitions are decided by the backend (`utils/orders.js` states that the backend owns the rules); an illegal transition is therefore rejected by the server and reported by the interface.

SCREENSHOT PLACEHOLDER

To capture: the /staff orders page, signed in as server@demo.example, with several orders and status filters

Figure 10. Staff order board

5.3 Administration interface

The administration area (`/admin`) is restricted to the admin role and has `AdminNav`. It contains eleven views and two shared components:

Table 9. Administration modules

Module	Views and components	Description
Dashboard	`AdminHomeView`, `AdminDashboardView`, `BarsChart`	Key indicators (value and change with an arrow and a text label) and a revenue bar chart drawn by the project's own `BarsChart` component, without a charting library.
Categories	`AdminCategoriesView`	Creation, editing and deletion of menu categories.
Menu	`AdminMenuView`, `AdminMenuItemNewView`, `AdminMenuItemEditView`, `MenuItemForm`	Menu item management with image upload; one shared form for creation and edition.
Settings	`AdminSettingsView`	Restaurant settings used by the whole application (name, currency, time zone).
Users	`AdminUsersView`, `AdminUserNewView`, `AdminUserEditView`	User list filtered by role and activity, creation and edition with roles client, staff and admin.
Reports	`AdminReportsView`	Sales reports with a CSV download.

These pages follow the same pattern as the customer lists: local state, a request counter, loading, empty and failed states, and service modules (`adminDashboardService`, `adminMenuService`, `adminUserService`, `adminReportService`, `settingsService`). Roles are checked by the guard for navigation and by the backend for every request.

SCREENSHOT PLACEHOLDER

To capture: the /admin dashboard signed in as admin@demo.example, showing the key indicators and the revenue chart

Figure 11. Administration dashboard

5.4 Cross-cutting implementation aspects

- **Accessibility.** Real buttons and links, labels tied to inputs, visible focus, `aria-live` regions for status messages, focus moved to the main region after navigation, and a skip to content through the layout.
- **Security.** No `v-html`; the whitelist of query values; `safeRedirect`; validated cart storage; tokens as described in Section 4.10.
- **Dead code.** `AreaPlaceholderView.vue` is not referenced by any route; it is a remnant of an earlier development step and does not affect the application.

Chapter 6 — Testing and Quality Assurance

6.1 Testing strategy

Tests are written together with the features, in four groups that follow the architecture: **unit** tests for pure helpers, stores, the router and the API layer; **component** tests for base and feature components; **view** tests for pages; and **service** tests for the staff and administration services. All tests run in Vitest with jsdom. Two helpers keep them uniform. `mountWithApp` mounts a component with a fresh Pinia, the real translations and the English locale. `fakeServer` replaces the Axios adapter with a function, records every request (method, URL, authorisation header, body) and lets the test answer like a real server, including error envelopes. Components are therefore tested with their real stores and translations but without any network.

6.2 Unit testing

Twenty-seven files in `tests/unit` cover the foundations. Examples include `money.test.js` (cent arithmetic), `menu-query.test.js` (URL whitelisting), `errors.test.js`, `session.test.js`, `i18n.test.js` (key parity) and `tokens-contrast.test.js` (WCAG contrast of the colour tokens in both themes).

6.3 Component testing

Twenty-six files in `tests/components` cover base components (`BaseButton`, `BaseInput`, `BaseModal`, `ConfirmDialog`, `ToastHost`), layout components (`AppHeader`, `AppFooter`), and the feature parts of the cart, menu, billing, queue and reservation pages. These tests check rendering, props, emitted events and accessibility attributes.

6.4 Store testing

Each store has a test file: `auth-store`, `cart-store`, `ui-store`, `notifications-store`, `restaurant-store` and `toasts-store`. They verify, for example, that the cart rejects tampered stored lines and computes totals in cents, that the authentication store restores a session once, and that the `ui` store persists the theme and the language.

6.5 Service testing

`tests/unit/api.test.js` verifies the Axios layer, and seven files in `tests/services` verify the staff and administration services. The API tests check, among other behaviours: that the access token is sent when present and left out of public requests; that failures become `ApiError` objects carrying the backend envelope; that a missing network is reported clearly; that an expired token is renewed and the request replayed with the new token; that several simultaneous failures trigger a single renewal; that the session ends, with one announcement, when the renewal fails; that requests which opt out are not renewed; and that a request is replayed only once, so that a rejection after renewal cannot loop.

6.6 Integration testing

Several test files cross layer boundaries and act as integration tests: `router-guards.test.js` (the guard with the real store and routes), `integration-nav.test.js`, `session-expiry.test.js`, `router-focus.test.js`, `staff-routes.test.js` and `cart-routes.test.js`. They verify that a visitor is sent to sign in with the way back remembered; that each role enters its own area; that an administrator may enter the staff area; that a stored session is re-opened before the page is decided; that a dead session does not trap the visitor; and that titles and focus follow the language and the navigation. The tests of the view groups (`tests/views`) also combine router, stores, services and templates. There are no browser-based end-to-end tests.

6.7 Test execution results

The suite was executed on a clean installation (`npm ci`, Node.js 22) for this report. The historical figures in the project documentation were not used; Table 10 gives the verified results.

Table 10. Verified quality results

Check	Command	Result
Automated tests	`npx vitest run`	85 test files, 735 tests passed, 0 failed, 0 skipped
Static analysis	`npx eslint .`	0 errors, 0 warnings
Production build	`npx vite build`	Successful; main chunk about 275 kB (95 kB gzip); views split into lazy chunks

Table 11. Test files by group

Test group	Files
`tests/unit`	27
`tests/components`	26
`tests/views`	25
`tests/services`	7
Total	85

The result shows that the main behaviours of the application are protected against regression, especially authentication, routing, the cart and the staff and administration services. It does not show that the interface is free of visual defects or that it works against the live backend: the tests use a simulated network and jsdom, not a real browser.

6.8 Code quality and linting

ESLint 10 runs with the recommended rules and the essential rules of `eslint-plugin-vue`. The configuration adds project rules (Appendix A.1): Options API only, restricted imports of the Composition API, explicit emits, required prop defaults, multi-word component names and no `v-html`. The architecture decisions therefore remain true as the code evolves, because a violation fails the check. The size of the code (about 13,600 lines in the Vue components, 2,300 lines of JavaScript in `src` and 7,800 lines of tests) shows that the tests are of the same order as the application code.

6.9 Known limitations of the quality process

- No code-coverage report is configured, so the percentage of code exercised by the tests is not measured.
- No browser end-to-end tests and no visual regression tests; the tests do not run in a real browser.
- The tests use mocked HTTP; compatibility with the live backend depends on the API contract being respected.
- No continuous-integration pipeline: the three checks are executed manually.
- Some staff services (`staffOrderService`, `staffPaymentService`) use a code style (semicolons, default export) different from the other services. It is cosmetic and does not affect behaviour.

Chapter 7 — Results, Limitations and Future Improvements

7.1 Achieved objectives

Table 12. Assessment of the specific objectives

Specific objective	Result
Components and lazy views	Achieved: 80 components, 43 routes, every view lazy-loaded.
Shared state with Pinia	Achieved: 8 stores, with a documented criterion separating shared and local state.
Navigation and access control	Achieved: global guard, role metadata, safe redirect, focus management.
API communication with token renewal	Achieved: one Axios instance, single-flight refresh, uniform error type.
English and French	Achieved for the interface; backend content is not translated.
Custom accessible design system	Achieved: tokens, scoped CSS, light and dark themes, contrast tests.
Automated verification	Achieved: 735 passing tests, clean lint, successful build.

7.2 Implemented functionality

The customer journey is implemented from the menu to invoices and notifications. The staff area provides seven views, and the administration area provides eleven views and two shared components. These features were verified in the source code and exercised by the tests; they were not taken from the planning documents. The project documentation described the staff and administration modules inconsistently (planned in one section, complete in another); the code shows that they are implemented.

7.3 Technical achievements

- A strict Options API code base enforced by the linter, with Pinia stores in the options style.
- A session design that keeps the access token out of persistent storage and renews it once for concurrent requests.
- A stale-response guard on every list page and a URL-driven filter state.
- A complete custom design system without a UI framework, with a tested contrast policy.
- A translation process that fails the build when a key is missing in one language.
- A test infrastructure that replaces the network and gives fast, deterministic tests.

7.4 Current limitations

- **Payments are simulated.** Card and mobile-money payments do not use a real payment gateway.

- **No production deployment.** The application is run locally with the backend; no public deployment exists at the time of writing.
- **Table scanning by URL only.** There is no camera-based scanner in the application.
- **Partial translation.** Menu content and notification messages come from the backend in a single language.
- **Test scope.** jsdom and mocked HTTP only; no coverage measurement, no end-to-end tests, no CI.
- **Token storage trade-off.** The refresh token remains readable by scripts in the page (`localStorage`).
- **No TypeScript.** Types of API data are documented by usage and tests rather than checked by the compiler.
- **Polling.** Order and notification updates use periodic requests instead of server-push channels, which is simple but adds traffic.
- **Dead code.** An unused placeholder view remains in the repository.

7.5 Future improvements

- Integrate a real payment provider behind the existing payment service interface.
- Deploy the frontend as a static site and the backend on a hosting service, with the API address set at build time and the frontend origin added to the backend CORS settings.
- Add a continuous-integration workflow running lint, tests and build, and a coverage report.
- Add end-to-end tests with a browser automation tool for the main journeys.
- Replace polling with WebSockets or server-sent events for orders, the queue and notifications.
- Translate menu content and notifications on the backend, for example by returning content per language.
- Add an in-app camera scanner for table QR codes.
- Consider TypeScript for the service layer and a progressive web application mode for offline resilience.

7.6 Lessons learned

Several lessons emerged. First, a restrictive architectural rule enforced by tooling (the Options API rule) is more reliable than a rule written in documentation. Second, the separation between shared and local state keeps stores small and pages easy to copy and test. Third, a single Axios layer concentrates the difficult parts of authentication (concurrency, rotation, expiry) in one place that can be tested thoroughly. Fourth, the project documentation must be checked against the code: the number of tests and the status of features differed between documents, which is why the figures of this report were obtained by running the project. Finally, designing without a UI framework requires discipline, and design tokens and shared base components provide that discipline.

General Conclusion

This project addressed the lack of integrated digital tools in restaurant operations: customers lack visibility on queues and orders, and staff and managers lack a shared view of orders, tables and figures. The solution is the frontend of a restaurant management application, written as a Vue 3 single-page application that connects to a Django REST API.

The main Vue.js technologies were used in a deliberate way. The Options API structures every component and is enforced by ESLint. Vue Router, with lazy views and a global guard, organises 43 routes and the access of each role. Pinia holds the eight stores that represent state shared between pages, while list pages keep their own state. Axios, configured once, provides authentication with an in-memory access token, a stored refresh token, a single-flight renewal and normalised errors. Vue I18n provides English and French with a test that guarantees equal keys. A custom CSS design system with tokens, scoped styles and two themes replaces a UI framework.

The objectives were achieved: the customer, staff and administration modules exist, and the application is verified by 735 passing tests, a clean lint run and a successful production build. The limitations are clear: payments are simulated, the application has not been deployed, backend content is not translated, and the tests do not cover real browsers or the live backend. The improvements listed in Chapter 7 (real payments, deployment, continuous integration, end-to-end tests and real-time updates) form a realistic next stage. The project demonstrates that a well-structured Vue application can be built with the framework's core tools alone, and that its quality can be protected by automation.

References

References follow the Harvard author–date style. The list contains only official documentation and standards that were used to understand the technologies of the project; no other source is cited. The project repositories are listed first.

CIRHULWIRE AJUWA, J. (2026) restaurant-frontend [source code]. Available at: https://github.com/monamijer/restaurant-frontend.

CIRHULWIRE AJUWA, J. (2026) restaurant-backend [source code]. Available at: https://github.com/monamijer/restaurant-backend.

Axios (n.d.) Axios documentation. Available at: https://axios-http.com/docs/intro.

ESLint (n.d.) ESLint documentation. Available at: https://eslint.org/docs/latest/.

eslint-plugin-vue (n.d.) eslint-plugin-vue documentation. Available at: https://eslint.vuejs.org/.

Fontsource (n.d.) Fontsource documentation. Available at: https://fontsource.org/docs/getting-started/introduction.

MDN Web Docs (n.d.) Web Storage API. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API.

MDN Web Docs (n.d.) CSS custom properties for cascading variables. Mozilla. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties.

Pinia (n.d.) Pinia documentation. Available at: https://pinia.vuejs.org/.

Vite (n.d.) Vite documentation. Available at: https://vite.dev/guide/.

Vitest (n.d.) Vitest documentation. Available at: https://vitest.dev/guide/.

Vue I18n (n.d.) Vue I18n documentation. Available at: https://vue-i18n.intlify.dev/.

Vue Router (n.d.) Vue Router documentation. Available at: https://router.vuejs.org/.

Vue Test Utils (n.d.) Vue Test Utils documentation. Available at: https://test-utils.vuejs.org/.

Vue.js (n.d.) Vue.js documentation. Available at: https://vuejs.org/guide/introduction.html.

W3C (2023) Web Content Accessibility Guidelines (WCAG) 2.2. Available at: https://www.w3.org/TR/WCAG22/.

Appendices

Appendix A — Configuration excerpts

A.1 ESLint rules enforcing the architecture (eslint.config.js)

COMPOSITION_API lists ref, reactive, computed, watch, onMounted, provide, inject, etc.

```
'vue/component-api-style': ['error', ['options']],
'no-restricted-imports': ['error', { paths: [{ name: 'vue',
    importNames: COMPOSITION_API, message: 'Options API only.' }] }],
'vue/multi-word-component-names': ['error', { ignores: ['App'] }],
'vue/require-default-prop': 'error',
'vue/require-explicit-emits': 'error',
'vue/no-v-html': 'error',
```

A.2 Environment and scripts

Table 13. Environment variable and scripts

Item	Value
`VITE_API_BASE_URL`	Base URL of the Django API (default `http://localhost:8000/api`)
`npm run dev`	Development server on port 5173 (`strictPort`)
`npm run build` / `preview`	Production build in `dist/` / local preview
`npm test` / `npm run lint`	Vitest suite / ESLint

Appendix B — Test helper

tests/helpers/http.js — replaces the network and records requests (abridged)

```
export function fakeServer(handler) {
  const requests = []
  api.defaults.adapter = async (config) => {
    requests.push({ method: config.method, url: config.url,
      auth: config.headers.Authorization, body: config.data })
    return handler(config)
  }
  return requests
}
```

Appendix C — Verified project statistics

Table 14. Verified statistics of the frontend repository

Measure	Value
Vue single-file components	80 (about 13,600 lines)
JavaScript in `src` (non-Vue)	about 2,300 lines
Routes (declared paths)	43 (28 require authentication)
Pinia stores / service modules / mixins / directives	8 / 22 / 2 / 1

Measure	Value
Translation namespaces	16 per language (en, fr)
Test files / tests	85 / 735 (0 failed, 0 skipped)
Lines of test code	about 7,800
Commits in the repository	92 (14 October – 5 November 2026)
Production main chunk	about 275 kB (95 kB gzip)

Appendix D — Screenshots to capture

The following screens should replace the placeholders. Sign in with the demo accounts created by the backend command `seed_demo` (`admin@demo.example`, `server@demo.example`, `client1@demo.example`), use a desktop width of about 1280 px, and capture both themes where a placeholder says so.

- Home page (Figure 1).
- Menu page with filters.
- Checkout with the simulated payment panel.
- Order tracking timeline.
- Staff order board.
- Administration dashboard.